

Cours 4 PAM

Build Beautiful Apps -> Material Design

Material Design : Façon de modéliser l'interface selon certains critères (mode sombre etc...)

On va aborder

- Les palettes de couleurs :
- La typographie
- Les shapes (modifier l'affichage de certains composables)

Pour chaque mode, il y'a des couleurs prédéfinies via une palette de couleur.

Palettes de couleurs Mode Clair :

Light Scheme

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Palettes de couleurs Mode Sombre :

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Typographie :

Type Role	Font	Font weight	Size
displayLarge	Abril Fatface	Normal	36sp
displayMedium	Montserrat	Bold	20sp
labelSmall	Montserrat	Bold	14sp
bodyLarge	Montserrat	Normal	14sp

Cette typographie va être stockée dans **Theme.kt** qui va définir la typographie, les couleurs, les shapes...

Color.kt : Ensemble d'instances de couleurs que l'on va initialiser ici

Shape.kt : Là où l'on va initialiser nos shapes.

Type.kt : Là où l'on va définir nos styles d'écriture, nos polices etc...

Avec **dimens.xml**, on peut y prédéfinir des paddings, ou bien des tailles d'image afin que toutes nos images fassent la même taille.

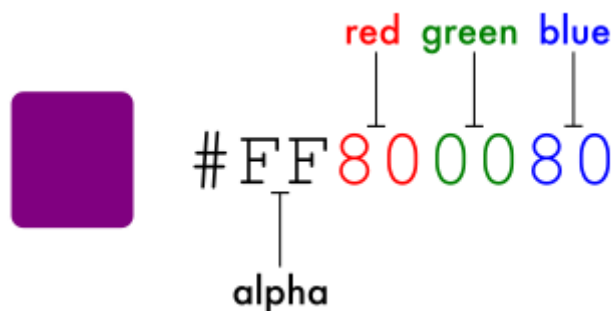
```
<resources>
  <dimen name="padding_small">8dp</dimen>
  <dimen name="padding_medium">16dp</dimen>
  <dimen name="image_size">64dp</dimen>
</resources>
```

Important à retenir :

```
text = stringResource("%d years old", ...formatArgs = dogAge),
```

Tel que %d est remplacé par **dogAge**

Modifier le color scheme :



A color can also include an alpha value—letters and/or numbers—which represents the transparency of the color (#00 is 0% opacity (fully transparent), #FF is 100% opacity (fully opaque)). When included, the alpha value is the first two characters of the hex color code after the pound (#) character. If an alpha value is not included, it is assumed to be #FF, which is 100% opacity (fully opaque).

Utiliser *Material Theme Builder* afin de pouvoir générer nous même une couleur et une police comme l'on souhaite.

<https://material-foundation.github.io/material-theme-builder/>

Dynamic color

Va générer automatiquement une palette clair/sombre.

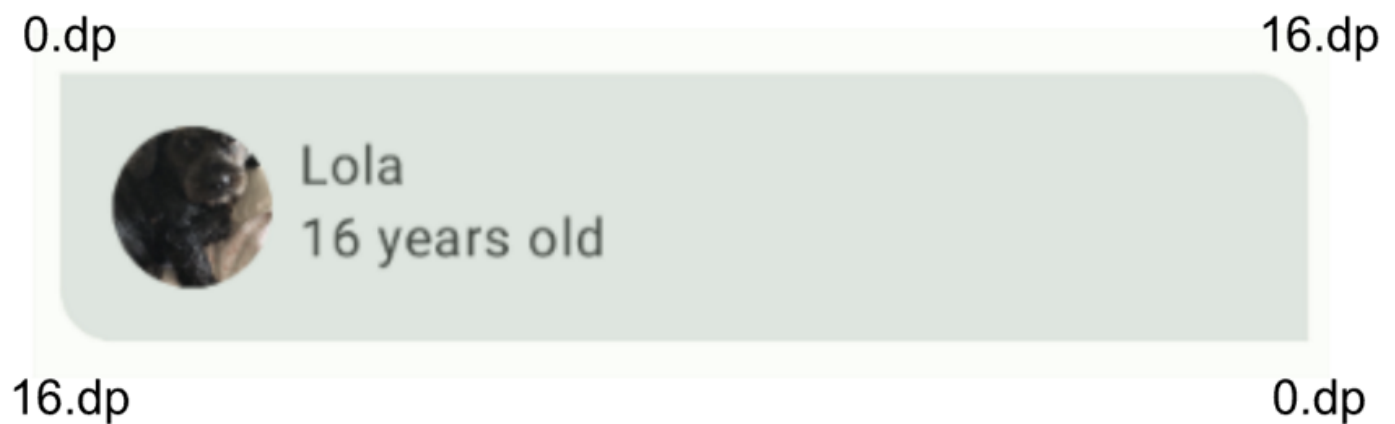
Shape : dans *Shape.kt*

RoundedCornerShape : Rectangle avec coins arrondis. Dans ses paramètres, on peut indiquer le degré d'arrondi.



RoundedCornerShape(0.dp) et 25.dp

On peut également modifier 1 à 1 le coin que l'on souhaite modifier en utilisant **top left top right, bottom left et bottom right**



Dans MainActivity, si jamais l'on veut attribuer une Shape à une image, on va utiliser dans le **modifier** la fonctionnalité **clip**.

```
Image(  
    modifier = modifier  
        .size(dimensionResource(id = R.dimen.image_size))  
        .padding(dimensionResource(id = R.dimen.padding_small))  
        .clip(MaterialTheme.shapes.small),  
    contentScale = ContentScale.Crop,
```

Contrôle *comment l'image s'adapte à la taille du conteneur (ici Image)*

```
Image(  
    modifier = modifier  
        .size(size = dimensionResource(64dp)) //Fixe une taille à notre image à 64dp  
        .padding(all = dimensionResource(8dp)) //Ajoute un padding à l'extérieur de notre image de 8dp  
        .clip(shape = MaterialTheme.shapes.small), //Applique un roundedCornerShape small  
    contentScale = ContentScale.Crop, //signifie que l'image est redimensionnée de manière à remplir tout le cadre défini par Size  
    painter = painterResource(id = dogIcon),  
  
    // Content Description is not needed here - image is decorative, and setting a null content  
    // description allows accessibility services to skip this element during navigation.  
  
    contentDescription = null  
)
```

Au lieu de définir directement dans nos fonctions le type de caractères que l'on souhaite (font, style etc...) On va créer des types de styles que l'on va appliquer directement à nos éléments (cf **Type.kt**)

Top Bar : Barre en haut de l'écran qui ne bouge pas

```
@Composable
fun WoofAppBar(modifier: Modifier = Modifier) {
    CenterAlignedTopAppBar(
        title = {
            Row(
                verticalAlignment = Alignment.CenterVertically
            ) {
                Image(
                    modifier = Modifier
                        .size(dimensionResource(id = R.dimen.image_size))
                        .padding(dimensionResource(id = R.dimen.padding_small)),
                    painter = painterResource(R.drawable.ic_woof_logo),

                    contentDescription = null
                )
                Text(
                    text = stringResource(R.string.app_name),
                    style = MaterialTheme.typography.displayLarge
                )
            }
        },
        modifier = modifier
    )
}
```

Et dans *WoofApp*, la mettre dans un *Scaffold* juste avant la *LazyColumn*

Il existe également la **composable** *CenteredHorizontallyTopAppBar*

```
import androidx.compose.material3.CenterAlignedTopAppBar

@Composable
fun WoofTopAppBar(modifier: Modifier = Modifier) {
    CenterAlignedTopAppBar(
        modifier = modifier
    )
}
```

Différence entre Image et une Icon

L'Icon ne définit qu'une forme monochrome et tu peux changer le background de ton Icon, ce qui n'est pas le cas d'une Image.

Voici la traduction du résumé :

Résumé

Le **Material Theming** te permet d'utiliser le **Material Design** dans ton application, avec des directives pour **personnaliser les couleurs, la typographie et les formes**.

- **Theme.kt** : c'est le fichier où le thème est défini, via une fonction composable nommée [nom de ton app]+Theme() — par exemple **WoofTheme()** pour cette application. À l'intérieur de cette fonction, l'objet **MaterialTheme** configure les **couleurs, la typographie, les formes et le contenu** de l'application.
- **Color.kt** : c'est ici que tu listes toutes les couleurs utilisées dans l'application. Ensuite, dans **Theme.kt**, tu assignes les couleurs aux **LightColorPalette** et **DarkColorPalette** pour les différents rôles (slots). Tous les rôles n'ont pas besoin d'être assignés.
- Ton application peut choisir d'activer le **Force Dark**, ce qui signifie que le système appliquera automatiquement un **thème sombre**. Cependant, il est préférable pour les utilisateurs que tu implémentes toi-même le thème sombre, afin de garder un **contrôle total sur le thème**.
- **Shape.kt** : c'est ici que tu définis les **formes de ton application**. Il existe trois tailles de formes : **small, medium, large**, et tu peux spécifier comment les coins sont arrondis. Les formes servent à **attirer l'attention, identifier les composants, communiquer l'état et exprimer l'identité de la marque**.
- **Type.kt** : c'est ici que tu initialises tes **polices** et que tu assignes **fontFamily, fontWeight** et **fontSize** pour l'échelle typographique de Material Design. L'échelle typographique de Material Design inclut une **gamme de styles contrastés** qui soutiennent les besoins de ton application et de son contenu. Elle combine **15 styles** pris en charge par le système typographique.